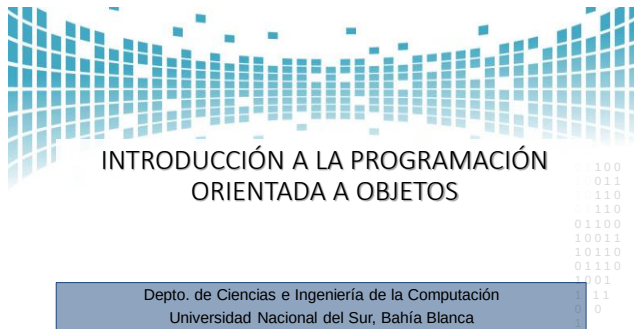




Colección	
HojaEnfermeria <<Atributos de instancia>> t [] SignosVitales cant: entero <<Constructores>> HojaEnfermeria(max: entero) <<Comandos>> insertar(s: SignosVitales) eliminar(s: SignosVitales) <<Consultas>> cantMediciones(): entero estaLlena(): boolean cantAlarmas(): entero hayDosAlarmasSeguidas(): boolean hojaAlarmas(): HojaEnfermeria Las primeras cant posiciones están ligadas, las demás no están ligadas	SignosVitales <<Atributos de clase>> umbralTemp = 38 <<Atributos de instancia>> temperatura: real presion: PresionArterial <<Constructor>> SignosVitales(t: real, p: PresionArterial) <<Consultas>> obtenerTemperatura(): real obtenerPresion(): PresionArterial alarma(): boolean equals(s: SignosVitales): boolean toString(): String

2

Colección

HojaEnfermeria(max: entero) crea una estructura con capacidad para mantener hasta max mediciones de signos vitales. Requiere max mayor a 0.

insertar(s: SignosVitales) Asigna s a la primer posición libre de la colección y aumenta la cantidad de posiciones ligadas. Requiere que s esté ligado y la clase cliente haya verificado que la colección no esté llena.

eliminar(s: SignosVitales) Busca la primera aparición de s en la colección por estado interno, si lo encuentra lo elimina y decrementa la cantidad de mediciones registradas. Requiere que s esté ligado.

Colección

cantMediciones(): entero retorna la cantidad de mediciones registradas.

estaLlena(): boolean retorna true si la colección está llena.

cantAlarmas(): entero retorna la cantidad de mediciones con alarma.

hayDosAlarmasSeguidas(): boolean retorna true si hay al menos dos mediciones consecutivas con alarma.

hojaAlarmas(): HojaEnfermeria Si hay mediciones con alarma retorna una nueva colección con las mediciones que produjeron alarma. En caso contrario retorna nulo.

Colección

Las clases HojaEnfermeria y SignosVitales están asociadas.

Los elementos de la clase HojaEnfermeria están "comprimidos". Esto es, los elementos ligados a objetos de clase SignosVitales ocupan las primeras cant posiciones del arreglo y todos los elementos nulos ocupan las posiciones que siguen.

Decimos que HojaEnfermeria modela una **colección** de elementos de clase SignosVitales.

El atributo cant mantiene en cada momento la cantidad de elementos ocupados en el arreglo y también la posición en la que se va a insertar el próximo elemento.

Colección

```

class HojaEnfermeria{
    //Atributos de Instancia
    private SignosVitales[] t;
    private int cant;
    //Constructor
    public HojaEnfermeria(int max){
        /*Crea una estructura con capacidad para max mediciones de
        signos vitales. Requiere max mayor a 0*/
        t = new SignosVitales [max];
        cant = 0;
    }
}
    
```

Adoptamos la convención de llamar **colección** a las estructuras que mantienen una cantidad fija de componentes ligadas ocupando las primeras posiciones y las demás libres.

Colección

```
//Comandos
public void insertar(SignosVitales s){
    /*Asigna s a la primer posiciones libre de la coleccion y aumenta
    la cantidad de posiciones ligadas. Requiere que s esté ligado y la
    clase cliente haya verificado que la coleccion no esté llena.*/
    t[cant] = s;
    cant ++;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Si cambia el compromiso entre las clases cambia la implementación del servicio.

Colección

HojaEnfermeria(max: entero) crea una estructura con capacidad para mantener hasta max mediciones de signos vitales. Requiere max mayor a 0.

insertar(s: SignosVitales) Si s está ligada y la colección no está llena, asigna s a la primer posición libre de la colección y aumenta la cantidad de posiciones ligadas. En caso contrario el comando no tiene ningún efecto.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El diseñador especifica explícitamente que el comando insertar debe realizar dos controles antes de asignar.

Colección

HojaEnfermeria(max: entero) crea una estructura con capacidad para mantener hasta max mediciones de signos vitales. Requiere max mayor a 0.

insertar(s: SignosVitales): boolean Si s está ligada y la colección no está llena, asigna s a la primer posición libre de la colección, y aumenta la cantidad de posiciones ligadas y retorna true. En caso contrario retorna false.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El resultado indica si se realizó la asignación.

Colección

HojaEnfermeria(max: entero) crea una estructura con capacidad para mantener hasta max mediciones de signos vitales. Requiere max mayor a 0.

insertar(s: SignosVitales) asigna s a la primer posición libre de la colección y aumenta la cantidad de posiciones ligadas.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El diseñador no especificó qué ocurre si s es nula o la colección está llena, debemos controlarlo para evitar resultados erróneos.

Colección

```
//Consultas
public int cantMediciones(){
    //Retorna la cantidad de mediciones registradas
    return cant;
}

public boolean estaLlena(){
    //Retorna true si la coleccion está llena
    return cant == t.length;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Colección

```
//Consultas
public int cantAlarmas(){
    //Retorna la cantidad de mediciones con alarma
    int contador = 0;
    for (int i = 0; i < cant; i++)
        if (t[i].alarma())
            contador++;
    return contador;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Colección

```
public boolean hayDosAlarmasSeguidas() {
    /*Retorna true si hay al menos dos mediciones consecutivas con
    alarma*/
    boolean dos = false;
    for (int i = 0; i < cant-1 && !dos; i++)
        dos = t[i].alarma() && t[i+1].alarma();
    return dos;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Colección

```
public HojaEnfermeria hojaAlarmas() {
    /*Si hay mediciones con alarma retorna una nueva colección con las
    mediciones que produjeron alarma. En caso contrario retorna nulo*/
    HojaEnfermeria h = null;
    int contador = cantAlarmas();
    if (contador > 0) {
        h = new HojaEnfermeria(contador);
        for (int i = 0; i < cant && contador > 0; i++)
            if (t[i].alarma()) {
                h.insertar(t[i]);
                contador--;
            }
        return h;
    }
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Colección

```
public void eliminar(SignosVitales s) {
    /*Busca la primera aparición de s en la colección por estado
    interno, si lo encuentra lo elimina y decrementa cant. Requiere
    que s esté ligado*/
    int pos = buscarPos(s);
    if (pos >= 0 && pos < cant) {
        col[pos] = col[cant-1];
        col[cant-1] = null;
        cant--;
    }
    private int buscarPos(Elemento e) {
        int toRet = -1; int i = 0;
        while (i < cant && toRet == -1) {
            if (col[i].equals(e))
                toRet = i;
            i++;
        }
        return toRet;
    }
}
```

```
01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1
```

15

La clase Empleado

Empleado
<<Atributos de instancia>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntigüedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

Empleado(leg: entero, sb: real, fi: Fecha) Requiere fi ligada
obtenerAntigüedad(hoy: Fecha): entero Requiere hoy ligada y posterior a fechaIngreso. Retorna la diferencia en años entre fechaIngreso y hoy.
obtenerVacaciones(hoy: Fecha): entero Requiere hoy posterior a fechaIngreso. Retorna la cantidad de días de vacaciones que le corresponden empleado. 7 si antigüedad mayor o igual a 1 y menor o igual a 5. 15 si antigüedad mayor a 5 y menor o igual a 10. 25 si antigüedad es mayor a 10. Si tiene menos de 1 año de antigüedad, no tiene vacaciones

La clase Empleado

Empleado
<<Atributos de instancia>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntigüedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

```
obtenerSueldoNeto(p: Planilla, hoy: Fecha): real
Requiere p ligada y hoy ligada y posterior a fechaIngreso
Retorna el sueldo neto del empleado calculado como el sueldo básico incrementado según un porcentaje dependiente de la antigüedad y obtenido de la planilla p. Si tiene 15 años o más el sueldo neto es el doble del básico
```

```
01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1
```

Los modificadores de acceso

```
class Empleado {
    private int legajo;
    private float sueldoBasico;
    private Fecha fechaIngreso;
    //Constructor
    public Empleado(int leg, float sb, Fecha fi) {
        legajo = leg;
        sueldoBasico = sb;
        fechaIngreso = fi;
    }
    //Comandos
    //Consultas
}
```

```
00
11
10
10
10
20
11
10
10
1
1
```

El método obtenerVacaciones

```
public int obtenerVacaciones(Fecha hoy){
    /*Requiere hoy ligada y posterior a fechaIngreso. Retorna la
    cantidad de días de vacaciones que le corresponden empleado.
    7 si antigüedad mayor o igual a 1 y menor o igual a 5.
    15 si antigüedad mayor a 5 y menor o igual a 10.
    25 si antigüedad es mayor a 10.
    Si tiene menos de 1 año, no tiene vacaciones */
    int a = obtenerAntigüedad(hoy);
    int v = 0;
    if (a > 10)
        v = 25;
    else
        if (a > 5) v = 15;
        else
            if (a >= 1) v = 7;
    return v;
}
```

Colección

```
public float obtenerSueldoNeto(Planilla p, Fecha hoy){
    /*Requiere p ligada y hoy ligada y posterior a
    fechaIngreso. Retorna el sueldo neto del empleado
    calculado como el sueldo básico incrementado según un
    porcentaje dependiente de la antigüedad y obtenido de la
    planilla p. Si tiene 15 años o más el sueldo neto es el
    doble del básico*/
    int a = obtenerAntigüedad(hoy);
    float sn = 0; float porc;
    if (a >= 15) sn = 2 * sueldoBasico;
    else {
        porc = p.obtenerPorcentaje(a);
        sn = sueldoBasico + (sueldoBasico * porc / 100);
    }
    return sn;
}
```

Colección

Terna
<<Atributos de instancia>> Desde, hasta: entero porcentaje: real
<<Constructor>> Par(d,h:entero,p:float)
<<Consultas>> obtenerDesde(): entero obtenerHasta(): entero obtenerPorcentaje(): real
El porcentaje se aplica para el intervalo de valores comprendidos entre desde y hasta.

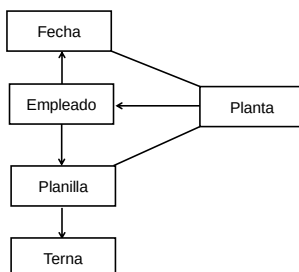
Planilla
t [] Terna cant: entero
<<Constructores>> Planilla(max: entero)
<<Comandos>> inserta(p: Terna) eliminar(p: Terna) <<Consultas>> cant(Ternas) estaLlena(): boolean solapados(d,h:entero): boolean obtenerPorcentaje(a:entero): real
Las primeras cant posiciones están ligadas, las demás no están ligadas.

Colección

Planilla(max: entero)
Requiere max mayor a 0
inserta(p: Terna) Agrega p a la colección si p está ligado y el intervalo no se solapa con otro de la colección
eliminar(p: Terna) Elimina, si existe, la terna p de la colección
solapados(d,h:entero): boolean Retorna true si la planilla mantiene una terna tal que desde <= d <= hasta o desde <= h <= hasta
obtenerPorcentaje(a:entero): real Busca una terna tal que desde <= a <= hasta y retorna el porcentaje asociado

Planilla
t [] Terna cant: entero
<<Constructores>> Planilla(max: entero)
<<Comandos>> inserta(p: Terna) eliminar(p: Terna) <<Consultas>> cant(Ternas) estaLlena(): boolean solapados(d,h:entero): boolean obtenerPorcentaje(a:entero): real
Las primeras cant posiciones están ligadas, las demás no están ligadas.

La relación entre clases



Colección

Empleado
<<Atributos de instancia>> legajo: entero sueldoBasico: real fechaIngreso: Fecha
<<Constructor>> Empleado(leg: entero, sb: real, fi: Fecha)
<<Comandos>> establecerSueldoBasico(sb: real)
<<Consultas>> obtenerLegajo(): entero obtenerSueldoBasico(): real obtenerFechaIngreso(): Fecha obtenerAntigüedad(hoy: Fecha): entero obtenerVacaciones(hoy: Fecha): entero obtenerSueldoNeto(p: Planilla, hoy: Fecha): real

Planta
<<Atributos de instancia>> t [] Empleado cant: entero
<<Constructores>> Planta(max: entero)
<<Comandos>> alta(e: Empleado) baja(leg: entero) aumentarBasicos(p: real)
<<Consultas>> cant(Empleados) estaLlena(): boolean estaEmpleado(leg: entero): boolean basicoRango(l1, l2: real): entero netoRango(l1, l2: real, p: Planilla, hoy: Fecha): entero masVacaciones(v: entero, hoy: Fecha): Planilla
Las primeras cant posiciones están ligadas, las demás no están ligadas.

Colección

Planta(max: entero) Requiere max mayor a 0
alta(e: Empleado) Requiere e ligado, la colección no está llena y no tiene un empleado con el mismo legajo que e.
estaEmpleado(leg: entero): boolean Retorna true si la colección tiene un empleado con legajo leg.
basicoRango(l1, l2: real): entero Computa cuántos empleados tienen el sueldo básico dentro del rango l1 y l2.
netoRango(l1, l2: real, p: Planilla, hoy: Fecha): entero Computa cuántos empleados tienen sueldos netos dentro del rango l1 y l2. Requiere p y hoy ligadas.
masVacaciones(v: entero, hoy: Fecha): Planta Genera una colección con los empleados con más de v días de vacaciones. Requiere hoy ligada.

Planta
<<Atributos de instancia>> t[] Empleado cant: entero
<<Constructores>> Planta(max: entero)
<<Comandos>> alta(e: Empleado) baja(leg: entero) aumentarBasicos(p: real)
<<Consultas>> cantEmpleados() estaLlena(): boolean estaEmpleado(leg: entero): boolean basicoRango(l1, l2: real): entero netoRango(l1, l2: real, p: Planilla, hoy: Fecha): entero masVacaciones(v: entero, hoy: Fecha): Planta
Las primeras cant posiciones están ligadas, las demás no están ligadas.

Colección

```
class Planta{
    //Atributos de instancia
    private Empleado[] t;
    private int cant;
    //Constructor
    public Planta(int max){
        /*Crea una colección con max elementos. Requiere max mayor a 0*/
        t = new Empleado [max];
        cant = 0;
    }
}
```

Colección

```
//Comandos
public void alta(Empleado e){
    /*Requiere e ligado, la colección no está llena y no tiene un empleado con el mismo legajo que e.*/
    t[cant] = e;
    cant++;
}
```

Colección

```
public Planta masVacaciones(int v, Fecha hoy){
    /*Genera una colección con los empleados con más de v días de vacaciones. Requiere hoy ligada*/
    Planta nueva = new Planta(cantEmpleados());
    for (int i = 0; i < cantEmpleados(); i++)
        if (t[i].obtenerVacaciones(hoy) > v)
            nueva.alta(t[i]);
    return nueva;
}
```

Colección

```
public int netoRango(float l1, float l2, Planilla p,
    Fecha hoy){
    /*Computa cuantos empleados tienen sueldos netos dentro del rango l1 y l2. Requiere p y hoy ligadas.*/
    int contador=0; float s;
    for (int i = 0; i < cantEmpleados(); i++){
        s = t[i].obtenerSueldoNeto(p, hoy);
        if (l1 <= s && s <= l2)
            contador++;
    }
    return contador;
}
```

La interpretación de la especificación

La interpretación de la especificación del **problema** forma parte de la resolución, así como también la verificación de la **solución** para un conjunto significativo de casos de prueba.

La consulta **c** retorna true si:

- La colección mantiene a lo sumo un elemento que verifica la propiedad
- La colección mantiene al menos un elemento que verifica la propiedad
- Existe un elemento en la colección que verifica la propiedad
- Existe un único elemento en la colección que verifica la propiedad

La interpretación de la especificación

Una de las competencias necesarias para resolver un problema en IPOO es entonces la comprensión lectora.

La consulta `c` retorna true si:

- La colección mantiene al menos una secuencia de al menos `n` elementos consecutivos que verifican la propiedad.
- La colección mantiene a lo sumo una secuencia de al menos `n` elementos consecutivos que verifican la propiedad.
- La colección mantiene exactamente una secuencia de al menos `n` elementos consecutivos que verifican la propiedad.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



El rol del diseñador

El diseñador establece el contrato entre las clases proveedoras y sus clientes.

Algunas modificaciones de diseño provocan cambios solo en una clase proveedora. Otras decisiones de diseño también afectan a sus clases clientes.

El encapsulamiento aspira a que cada clase pueda ser utilizada como una caja negra, de modo que al modificarse tenga el menor impacto posible sobre las clases relacionadas.



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



El responsable de testing

El responsable de testing verifica en primer lugar cada módulo de software y luego valida que el contrato se cumple al verificar el sistema completo.



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

